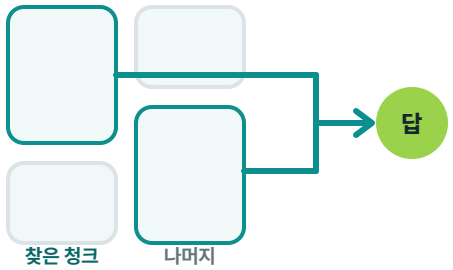
01 보충 교재 · SUPPLEMENT

RAG
검색해서 넣기

RAG = Retrieval-Augmented Generation(검색 증강 생성). 답을 만들기 전에 관련 문서 조각을 찾아(검색) 입력에 덧붙여(증강) 그 근거로 답하게(생성) 하는 방법

문서를 전부 넣는 대신 질문에 맞는 부분만 찾아 넣는 방법. 읽고 나면 research/ 노트 수백 건에 근거 경로가 붙은 답을 받을 수 있고, 벡터 DB가 언제 필요한지 스스로 판단할 수 있음



용어 먼저 — 이 교재에 나오는 말

처음 보는 말은 여기서 한 번 읽고 시작. 모르는 말이 나오면 이 장으로 돌아오기 — 맨 윗줄이 이 교재의 주제 용어, 아래로 갈수록 앞서 배운 말

용어	풀네임(영문)	한글 뜻	한 줄 설명	처음 나오는 장
RAG	Retrieval-Augmented Generation	검색 증강 생성	답을 만들기 전에 관련 문서 조각을 찾아(검색) 입력에 덧붙여(증강) 그 근거로 답하게(생성) 하는 방법	표지 · 5장
청크	Chunk	문서 조각	검색하기 좋게 문서를 잘라 둔 단위	5 · 6장
임베딩	Embedding	뜻을 담은 숫자 목록	글을 숫자 벡터로 바꾼 것. 뜻이 비슷하면 숫자도 가까움	5 · 7장
벡터 DB	Vector Database	벡터 저장소	임베딩을 저장하고 "가장 가까운 것"을 빠르게 찾아 주는 DB. pgvector(Supabase 확장) · Chroma · FAISS가 예	3 · 7장
그라운딩	Grounding	근거 고정	답에 쓰는 사실을 주어진 자료 안의 것으로만 제한하는 것	8장
할루시네이션	Hallucination	그럴듯한 오답	모델이 사실과 다른 내용을 자신 있게 생성하는 현상	8장
LLM	Large Language Model	대규모 언어 모델	다음에 올 토큰을 확률로 예측해 글을 생성하는 모델. Claude · ChatGPT · Gemini의 바탕	4장
토큰	Token	글자 조각	모델이 글을 세는 단위. 한국어는 글자 1~2개, 영어는 단어 조각 하나 정도	4장
컨텍스트 창	Context Window	입력 한도	모델이 한 번에 읽을 수 있는 토큰의 최대 분량. 대화 · 파일 · 지시가 모두 여기에 들어감	4장
API	Application Programming Interface	프로그램끼리의 창구	화면 없이 프로그램이 다른 프로그램(모델 · DB · 서비스)에 요청하고 답을 받는 규격	3장
Supabase	—	클라우드 DB 서비스	이 과정의 앱이 쓰는 데이터베이스 · 로그인 · 파일 저장 서비스(Day 25부터). products · orders · copy_log 테이블	7장

앞서 배운 말 — Claude Code(앤트로픽의 코딩 에이전트 — 이 과정의 유일한 AI 도구) · CLAUDE.md(Claude Code가 세션 시작 때 자동으로 읽는 규칙 파일) · 프롬프트(Prompt, 요청문) · md 파일(Markdown, 서식 있는 텍스트 문서) · DB(Database, 데이터베이스) · spec.md(요구사항 문서) · 서브에이전트(Sub-agent, 별도 컨텍스트의 조수) · LangChain(개발자용 LLM 앱 라이브러리 — 비교 대상으로만 등장)

※ 용어 뜻은 11개 보충교재 공통 (supplement_glossary 기준). 본문에서 처음 쓰는 자리에도 한 번 더 풀어 씀

이 교재의 위치 — 세 장 사이의 빈칸

폴더 전체를 읽히는 법(Day 6), 텍스트가 벡터가 되는 원리(Day 19), DB 정보만 근거로 답하는 설계(Day 25)는 배웠으나, 셋을 잇는 "필요한 부분만 찾아 넣기"의 이름과 절차가 비어 있음

이미 한 것	장 제목	그때 배운 것	산출물
Day 6 · 13장	Claude Code 인벤토리 생성 시연 — 폴더 전체를 읽고 표를 만들기	`research/` 전체를 읽혀 집계 — 문서가 수십 건일 때의 방법	`inventory/aggregate.md`
Day 19 · 7장	토큰화와 임베딩 — 텍스트가 숫자가 되는 경로	뜻은 벡터 사이의 거리로 표현 — "줄 길다"와 "웨이팅"이 묶이는 근거	원리 이해
Day 19 · 17장	컨텍스트 열화 — 긴 대화에서 성능이 떨어지는 이유	입력이 길수록 정확도 하락 — 필요한 것만 넣는 것이 원칙	세션 관리 규칙
Day 25 · 13장	AI API 연동 — 앱 안의 AI 기능, 서버에서만 호출	AI API(Application Programming Interface, 프로그램끼리의 창구) 호출 — 서버 함수(서버 쪽 코드)가 DB(Database, 데이터베이스)에서 상품 정보를 조회해 프롬프트(요청문)에 넣고, DB에 있는 정보만 근거로	`products` 조회 → 추천 사유

1 이름과 절차

"필요한 부분만 찾아 넣기"의 업계 이름이 **RAG**(Retrieval-Augmented Generation, 검색 증강 생성). 네 단계가 우리 수업 어디에 이미 있는지

이 덱에서

4 · 5장 — 한계와 네 단계

2 판단 기준

파일 검색으로 충분한 경우와 **벡터 DB**(Vector Database, 벡터 저장소)가 **필요해지는 조건**(문서 수 · 사용자 · 속도). 채용 공고의 "RAG 파이프라인"이 우리 어느 파일에 대응하는지

이 덱에서

6 · 7 · 9 · 10장 — 청킹 · 검색 방식 · 조건 · 대비표

3 파일로 남는 규칙

답마다 **근거 파일 · 문단을 붙이고, 못 찾으면 보류**하는 규칙을 `CLAUDE.md`와 `spec.md`(요구사항 문서)에 적는 실습 — 더미 데이터로 **적용 전 · 후를 나란히** 비교

이 덱에서

8 · 15~19장 — 규칙 문장과 실습 ① ② ③ · **11~14장** 기술 구축

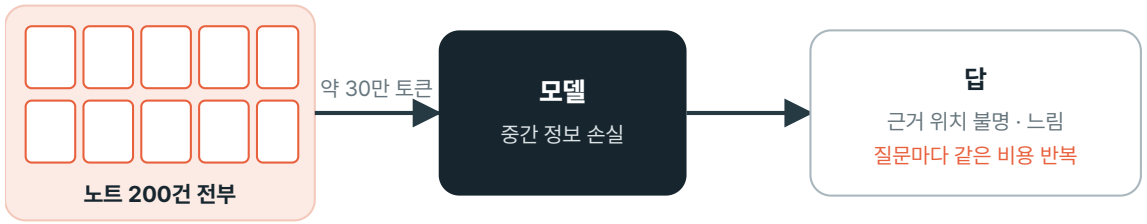
언제 읽으면 좋은가 — **Day 19 이후** 아무 때나. 실습 ①은 Claude Code만으로 가능, 실습 ② ③(벡터 검색 · 앱 명세)은 **Day 25 이후**

전부 넣기의 한계 — research/ 노트 200건을 다 넣으면

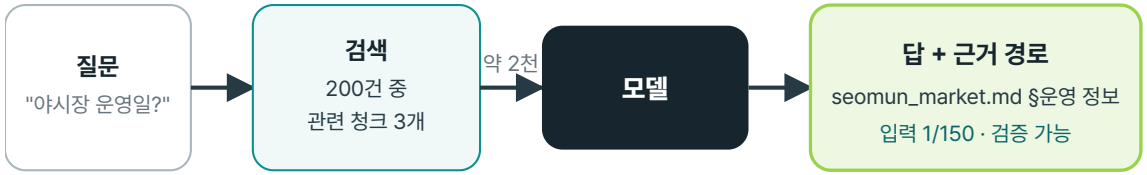
왜 알아야 하나 — "폴더 전체를 읽어 줘"는 문서 수십 건까지의 방법. 수백 건부터는 열화와 비용이 동시에 오고, 그 해법이 RAG

같은 질문, 두 가지 입력 방식

A · 전부 넣기



B · 찾아서 넣기 (RAG)



※ 개념도 · 토큰 수는 아래 표의 가정값

Day 19 산식에 대입 — 질문 1개의 입력 토큰(Token, 글자 조각 — 모델이 글을 세는 단위) · 가정값

항목	A · 전부	B · RAG
시스템 지시 + 질문	700	700
참고 자료 — 노트 1건 ≈ 1,500 토큰 가정	200건 × 1,500 = **300,000**	청크 3개 ≈ **1,200**
입력 합계 → 입력 단가 $2 / 100만 토큰	**약 $0.60**	**약 $0.004**
하루 질문 20개 × 월 22일	**약 $264**	**약 $1.7**

※ 노트 길이 · 질문 수는 가정, 단가는 Day 19 18장과 같은 Claude Sonnet 5.5 표준가 · 캐시 할인 미적용 · 2026.10 기준 · 수업 전 재확인

비용보다 먼저 오는 문제 — 열화

- **중간 정보 손실** — 관련 노트가 200건의 중간에 있으면 가장 못 씀 (Liu 외 2023, "Lost in the Middle" — 긴 입력의 중간 정보를 모델이 놓친다는 것을 보인 논문)
- **컨텍스트 창 한도** — 컨텍스트 창(Context Window, 입력 한도)은 LLM(Large Language Model, 대규모 언어 모델)이 한 번에 읽는 최대 토큰 수. 30만 토큰은 모델에 따라 아예 들어가지 않고, 들어가도 "잘 쓴다"와는 다름 (Day 19 17장)
- **근거 추적 불가** — 답이 어느 노트에서 왔는지 사람이 확인할 수 없음

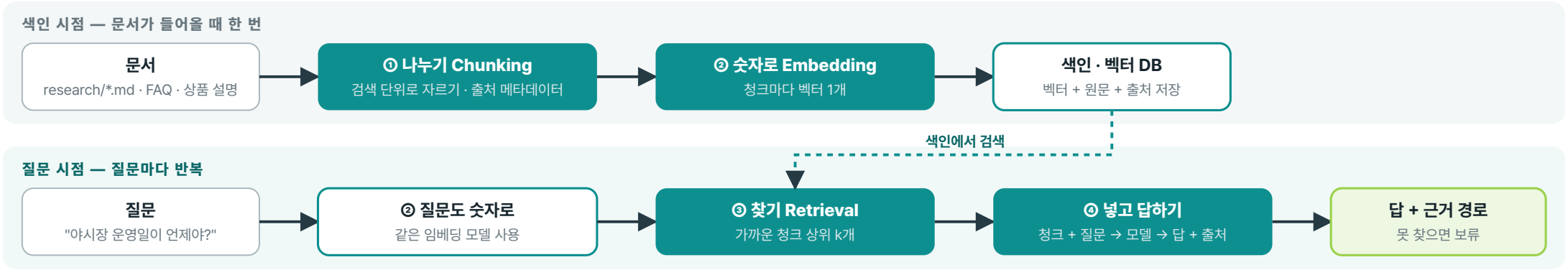
문서가 많아질수록 할 일은 **더 큰 컨텍스트 창을 찾는 것이 아니라 더 잘 고르는 것** — 고르는 절차의 이름이 RAG ※ arxiv.org/abs/2307.03172

RAG 네 단계 — 나누기 · 숫자로 · 찾기 · 넣고 답하기

왜 알아야 하나 — 네 단계 모두 우리 수업에 이미 흩어져 있음. 이름을 붙이면 채용 공고의 "RAG 파이프라인 구축"이 무엇을 묻는지 보임

정의 **RAG** `Retrieval-Augmented Generation · 검색 증강 생성` — 모델이 답을 생성하기 전에 외부 문서 저장소에서 질문과 관련된 문서 조각을 **검색(retrieve)**해 입력에 **덧붙이고(augment)**, 그 조각을 근거로 **생성(generate)**하는 방식

비유 도서관 — 책을 통째로 외우는 대신 색인카드를 만들어 두고(①②), 질문이 오면 카드로 서가에서 찾아(③) 그 쪽만 펴서 답함(④)



단계	하는 일	우리 수업에서 이미 한 곳	우리 파일
① 나누기	문서를 검색 단위(청크)로 자르고 출처 메타데이터를 붙임	Day 5 아카이빙 — 자산 1건 = 노트 1건, 제목 단위 절	`research/<유형>/<자산>.md`
② 숫자로	청크와 질문을 같은 임베딩 모델로 벡터화	Day 19 7장 — 토큰 → 벡터, 뜻은 거리	원리 (파일 없음)
③ 찾기	질문 벡터와 가까운 청크 k개, 또는 키워드 일치 파일	Day 6 13장 — Claude Code가 폴더를 읽고 집계 (파일 검색)	`Grep` (파일 안 글자 검색 도구) · `Read` (파일 읽기 도구) 호출
④ 넣고 답하기	찾은 청크만 프롬프트에 넣고, 그 안의 정보만 근거로 답	Day 25 13장 — 서버 함수가 DB 조회 결과를 프롬프트에 조립	`products` 조회 → 추천 사유

※ Lewis 외(2020) — RAG를 처음 제안한 논문. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", arxiv.org/abs/2005.11401 · 검색기(retriever)와 생성기(generator)를 결합. 단계 이름은 수업용 번역

① 청킹 — 문서를 자르는 크기와 겹침

보충 01 · 개념

왜 알아야 하나 — 검색은 청크 단위로 일어남. 잘못 자르면 그 뒤의 임베딩 · 검색 · 답이 모두 틀림. 우리 노트는 쓸 때 이미 절반쯤 잘려 있음

정의 **청크** Chunk · 문서 조각 — 검색과 임베딩의 단위가 되도록 문서를 일정 크기로 자른 조각. **크기**(chunk size)와 이웃 청크와의 **겹침**(overlap)을 정해 자르고, 조각마다 출처(파일 · 절 · 줄)를 메타데이터로 붙임

우리 예 서문시장 노트 1건(Day 2 29장의 `research/industry_food/seomun_market.md`)을 절 단위로 자르면 청크 3개

```
# 서문시장 야시장                ← 노트 1건 = 파일 1개
## 개요 · 운영 정보              ← 청크 1
위치 대구 중구 · 운영 금·토 저녁 …
## 리뷰 요약 (Day 4 수집)        ← 청크 2
"웨이팅 길어요" · "줄 길다" → 대기 태그 …
## 출처 · 미확인 항목            ← 청크 3
A 공식 2건 · C 언론 1건 · 미확인: 동절기 운영 …
```

청크	내용	메타데이터 (출처 표기의 근거)
1	개요 · 운영 정보	`seomun_market.md` · §개요 · 3~9행
2	리뷰 요약	`seomun_market.md` · §리뷰 요약 · 10~22행
3	출처 · 미확인 항목	`seomun_market.md` · §출처 · 23~30행

판단 기준 — 어떻게 자르나

1. **의미 단위로** — 제목(##) · 문단 경계에서 자름. 글자 수로만 자르면 표 한가운데가 끊김
2. **너무 작으면** 맥락 소실 — "금·토 저녁"만 남고 무엇의 운영일인지 사라짐
3. **너무 크면** 4장의 문제 재발 — 청크 하나가 노트 전체면 RAG가 아님
4. **겹침** — 앞 청크 끝 한두 문장을 다음 청크에 반복해 경계에서 잘리는 문장 방지
5. **메타데이터 필수** — 파일 · 절 · 줄 범위가 없으면 8장의 근거 표기가 불가능

Claude Code에서는

md의 `##` 절이 자연스러운 청크. `Grep`이 찾은 줄의 앞뒤 몇 줄을 함께 읽히는 것이 "겹침"의 역할. 노트를 쓸 때 절 제목을 질문 단위로 붙이는 것이 가장 싼 청킹

※ 크기의 정답은 없음 — 문서 유형별로 실험해 정함. 원 논문(Lewis 외 2020)은 위키피디아를 100단어 단위 조각으로 분할. 청킹 · 겹침 용어는 LangChain(개발자용 LLM 앱 라이브러리) 문서의 Text Splitters 기준 (python.langchain.com/docs/concepts/text_splitters)

② 임베딩 · 벡터 DB — "의미가 가까우면 벡터도 가까움"을 검색에 쓰기

보충 01 · 개념

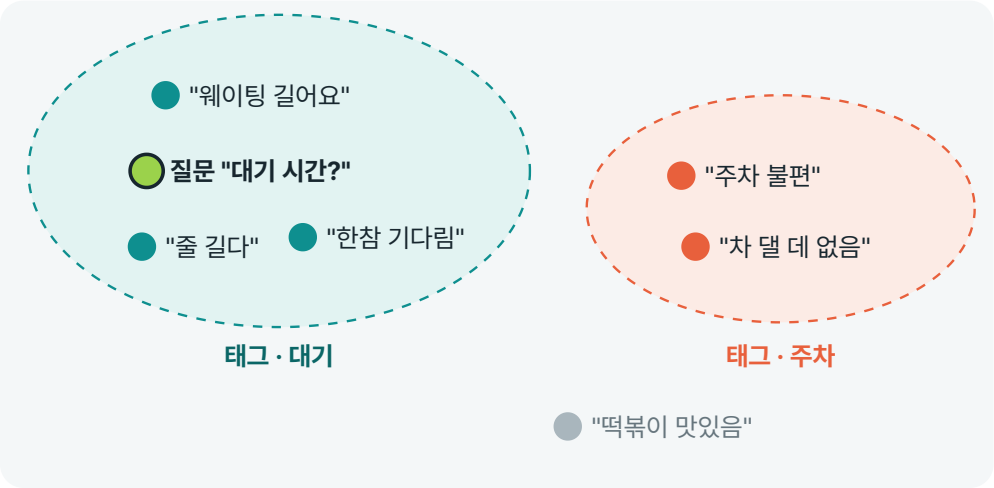
왜 알아야 하나 — 키워드 검색은 "웨이팅"으로 "줄 길다"를 못 찾음. 어느 검색이 어디에 맞는지 알아야 9장의 "벡터 DB가 필요한가"를 판단할 수 있음

정의

임베딩 `Embedding · 뜻을 담은 숫자 목록` — 텍스트를 의미가 보존되는 고정 길이의 숫자 벡터로 바꾼 것(Day 19 7장). **벡터 DB** `Vector Database · 벡터 저장소` — 청크의 벡터를 저장하고, 질문 벡터와 **거리가 가까운** 벡터를 빠르게 찾아 주는 데이터베이스

비유 지도 위의 핀 — 뜻이 비슷한 문장은 가까운 자리에 꽂힘. 검색은 "질문 핀에서 가장 가까운 핀 k개"를 집는 일

벡터 공간 — Day 4 리뷰 태그가 묶인 이유



※ 개념도 · 실측 데이터 아님. 실제 벡터는 수백~수천 차원, 그림은 2차원으로 압축한 이미지

	키워드 검색	의미(벡터) 검색
원리	문자열 일치 — 질문의 단어가 문서에 있는가	벡터 거리 — 질문과 뜻이 가까운가
"대기 시간?"	"대기"가 없는 "줄 길다" 노트 ✕	"줄 길다" · "웨이팅" 모두 ○
강점	고유명사 · 파일명 · 숫자 · 코드처럼 **정확히 그 글자**를 찾을 때	표현이 달라도 · 오타 · 사투리 · 긴 서술형 질문
약점	동의어 · 바꿔 말하기를 놓침	고유명사 · 숫자에 약함, 왜 골랐는지 설명 어려움, 색인 비용
도구	Claude Code `Grep` · DB `LIKE` · 전문 검색	Supabase(이 과정 앱의 클라우드 DB 서비스)의 `pgvector`(PostgreSQL 벡터 확장) · Chroma(오픈소스 벡터 DB)

실무의 흔한 구성은 둘 다 — 고유명사 · 상품 코드는 키워드로, 서술형 질문은 벡터로 찾고 합침(하이브리드 검색). 하나만 고르는 문제가 아님

※ 임베딩 정의는 Day 19 7장과 동일. pgvector는 PostgreSQL 확장 — supabase.com/docs/guides/ai · Chroma — docs.trychroma.com · 2026.10 기준 · 수업 전 재확인

④ 근거 표기와 보류 — 답에 출처 청크 붙이기, 못 찾으면 "모름"

왜 알아야 하나 — 검색이 틀리면 생성도 틀림. 답에 붙은 근거 경로가 사람이 검증할 수 있는 유일한 손잡이이고, 보류 규칙이 없으면 모델은 빈칸을 지어서 채움 — 할루시네이션(Hallucination, 그럴듯한 오답)

정의 **그라운딩** `Grounding · 근거 고정` — 답의 모든 주장이 검색된 청크 안의 정보에만 기대도록 묶는 것. 겉으로는 **출처 표기**(citation)와 근거가 없을 때의 **보류**(abstain)로 나타남

우리 예 Day 25 13장 "상품 정보에 없는 효능 · 원산지는 쓰지 말 것" · Day 2 출처 등급 A 공식 · B 공공 · C 언론 · D 개인이 그대로 근거 등급

답의 형식 — 질문 "서문시장 야시장 운영일은?"

답 금요일 · 토요일 저녁 운영. 동절기 운영 여부는 노트에 "미확인"으로 표시되어 있음

근거 `research/industry_food/seomun_market.md` §개요 · 운영 정보(5행) · 등급 A

등급 A 공식 1건 · 미확인 항목 1건 (동절기)

같은 형식, 근거 없음 — "야시장 주차장 요금은?"

답 보류 — `research/` 안에서 주차 요금 정보를 찾지 못함

확인함 `seomun_market.md` 전체, `Grep "주차"` 결과 2건(리뷰 "주차 불편" — 요금 언급 없음)

검색 결과	처리 규칙
근거 있음	답 + 파일 경로 · 절 · 행 + 출처 등급
근거 상충	둘 다 병기 + 각각의 경로 · 등급. 하나를 몰래 버리지 않음
근거 없음	**보류** + 무엇을 어디까지 찾았는지. 일반 지식으로 메우지 않음

CLAUDE.md에 적는 규칙 문장 — 16장 실습 ①에서 추가

```
## 노트 질문 답변 규칙
- 근거 폴더(기본 research/, 요청에서 지정하면 그 폴더) 안의 파일만 근거로 답한다
- 답마다 근거 파일 경로와 절 제목, 출처 등급(A~D)을 붙인다
- 근거를 못 찾으면 "보류"라고 쓰고, 확인한 파일과 검색어를 적는다
- 일반 지식으로 빈칸을 채우지 않는다
```

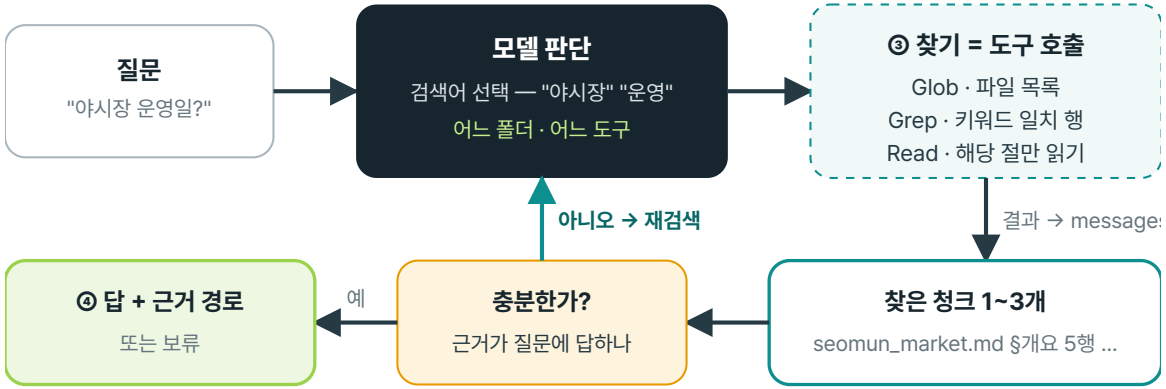
보류는 실패가 아니라 산출물 — **"무엇이 노트에 없는가"**가 다음 조사 대상 목록. Day 2 리서치 노트의 "확인 필요" 항목과 같은 자리

Claude Code에서의 RAG — 벡터 DB 없이, 파일 검색 도구 호출이 곧 Retrieve

보충 01 · 개념

왜 알아야 하나 — 우리는 이미 RAG를 쓰고 있음. 모델이 검색어를 고르고 Grep · Read를 부르는 것이 ③ 찾기. 벡터 DB는 특정 조건에서만 추가되는 부품

에이전트형 검색 — Day 2 31장 루프 안의 ③ 찾기



벡터 DB와 다른 점

· 검색어를 사람이 아니라 모델이 고르고, 결과를 보고 다시 고침 (에이전트형 검색)
· 키워드 일치라 "줄 길다"를 "대기"로 못 찾을 수 있음 → CLAUDE.md에 동의어 · 태그 규칙을 적어 보완

조건	파일 검색으로 충분	벡터 DB가 필요
문서 수	수십 ~ 수백 건의 md · csv	수천 건 이상, 계속 늘어남
누가 묻나	팀원이 Claude Code 안에서	**앱 사용자**가 화면에서 — Claude Code가 없는 곳
속도	도구 호출 몇 번의 시간 허용	화면 응답 안에 끝나야 함
질문 표현	노트의 용어를 아는 사람의 질문	표현이 제각각인 불특정 사용자
우리 수업	research/ 노트 Q&A, 기획 자료 조회	Day 25 앱의 "상품 Q&A" 기능 (11~14 · 19장)

세 가지가 "RAG 설정"

- 노트의 절 제목 = 청킹
- CLAUDE.md의 검색 · 근거 · 보류 규칙 = 검색기 설정
- Supabase `pgvector`는 앱 안 기능이 생길 때 추가 — 11~14장

※ Claude Code 내장 도구 Glob(이름 패턴으로 파일 찾기) · Grep · Read 명칭과 동작 · 2026.10 기준 · 수업 전 재확인. 벡터 DB 필요 조건은 수업용 판단 기준 (공식 수치 아님)

개발자 과정 vs 이 과정 — Load · Split · Embed · Retrieve는 우리 어느 파일인가

LangChain 튜토리얼의 RAG 파이프라인 여섯 부품과 우리가 만든 파일의 1:1 대응. 채용 공고의 용어는 왼쪽, 우리가 보여 줄 증거는 오른쪽

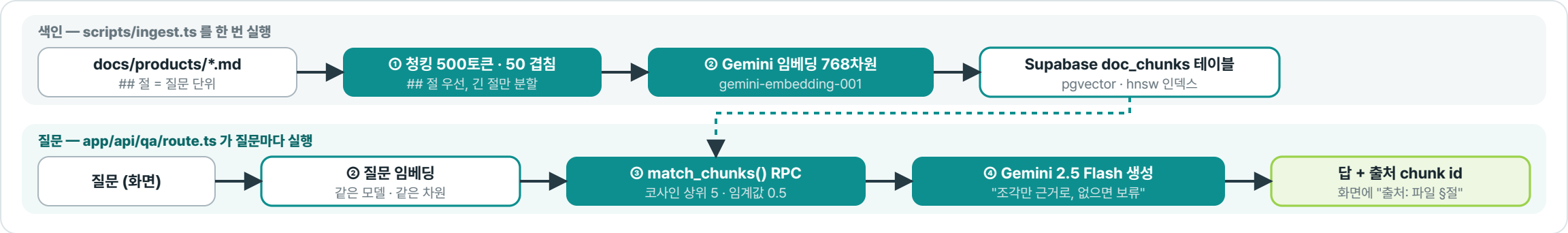
개념	LangChain · LangGraph 과정에서	이 과정(Claude Code)에서	우리가 만든 파일
Load 문서 적재	Document Loader로 PDF · 웹 · md를 Document 객체로	Day 2~5에 사람과 Claude Code가 직접 쓴 md 노트 · 공개 데이터 csv	`research/` · `archive/metadata.csv`
Split 청킹	Text Splitter — 크기 · 겹침을 숫자로 지정	노트의 `##` 절 = 청크. 쓸 때 질문 단위로 절을 나눔	노트 템플릿의 절 구조
Embed 임베딩	Embeddings 모델 호출 → 벡터	파일 검색 단계에서는 없음. 앱 기능이 생기면 Supabase `pgvector`로	`spec.md` §상품 Q&A (선택)
Store 벡터 저장	Chroma · FAISS(메타의 벡터 검색 라이브러리) 등 Vector Store	파일 시스템 + Git. 앱에서는 Supabase 테이블 + `pgvector`	팀 저장소 · `products` 테이블
Retrieve 검색	Retriever — similarity · k 값 설정	모델이 고른 검색어로 `Grep` · `Read` 호출. 앱에서는 서버 함수의 DB 조회	`CLAUDE.md` 검색 규칙
Generate 생성	Prompt Template + LLM 체인	Claude Code 응답 + 근거 · 보류 규칙. 앱에서는 서버 함수의 프롬프트 조립(Day 25 13장)	`CLAUDE.md` 규칙 · 서버 함수

같은 개념, 다른 도구 — **채용 공고의 용어는 왼쪽, 우리가 보여 줄 증거는 오른쪽.** "RAG 파이프라인을 구축했는가"에 대한 답은 노트 구조 · CLAUDE.md 규칙 · spec.md 단락 세 파일

※ 왼쪽 열은 LangChain 공식 RAG 튜토리얼의 구성 요소 명칭 (python.langchain.com/docs/tutorials/rag) · 2026.10 기준 · 수업 전 재확인. 이 과정에서 LangChain은 비교 대상으로만 등장

기술 구축 ① — RAG 아키텍처, 구성요소마다 무엇을 골랐나

실제로 어떻게 만드나 — Day 25 앱 스택 그대로: 문서는 md, 저장 · 검색은 Supabase의 PostgreSQL + pgvector 확장, 임베딩 · 생성은 Gemini API, 서버 함수는 Next.js. 벡터 DB 없는 길은 14장



구성요소	선택한 기술	왜 이것인가	대안
문서	`docs/products/*.md` — `##` 절 제목 = 질문 단위	사람이 읽고 고칠 수 있는 텍스트, Git 이력, 절 제목이 곧 출처 표기	DB 텍스트 열 · PDF(추출 단계 추가)
청킹	`##` 절 우선, 절이 길면 500토큰 · 50 겹침으로 분할	질문 단위 = 절. 500토큰이면 조각 5개를 넣어도 2,500토큰 — 4장의 30만 토큰과 비교	고정 길이만 · 문장 단위
임베딩	Gemini `gemini-embedding-001`, `outputDimensionality: 768`	Day 25와 같은 키 · 무료 등급, 차원을 줄여 인덱스가 작아짐. 질문과 문서에 반드시 같은 모델	OpenAI text-embedding-3 · 오픈소스 모델 (bge 등)
저장 · 검색	Supabase PostgreSQL + `pgvector` 확장, `hnsw` 인덱스, 코사인 거리	이미 쓰는 DB에 확장만 켜면 됨 · products 테이블과 join 가능 · 서비스 추가 없음	Chroma · Pinecone(별도 서비스)
검색 함수	SQL 함수 `match_chunks()`를 RPC로 호출	k · 임계값을 한 곳에서 관리, 서버 함수에서 한 줄 호출	앱 코드에서 SQL 직접 작성
생성	Next.js(App Router) Route Handler + Gemini 2.5 Flash	Day 25 13장 구조 그대로 — 키는 서버에만, 출력 형식 고정, 근거 제한	Supabase Edge Function · n8n 노드

※ 모델 이름(gemini-embedding-001 · gemini-2.5-flash) · 차원 옵션 · 무료 등급 한도는 ai.google.dev 기준, pgvector · hnsw는 supabase.com/docs/guides/ai 기준 · 2026.10 기준 · 수업 전 재확인. RPC(Remote Procedure Call) = DB 안의 함수를 앱에서 이름으로 부르는 것

기술 구축 ② — STEP 1~3, 저장소 · 적재 · 검색 함수

수강생은 타이핑하지 않음. 노란 줄이 Claude Code에 보내는 요청, 회색 주석이 "왜 이 코드가 생기는가". 실제로 동작하는 최소 형태 — 전체 파일은 Claude Code가 완성

STEP 1 · 확장 · 테이블 · 인덱스 — Supabase SQL Editor

```
-- 벡터 확장 켜기 (프로젝트당 한 번)
create extension if not exists vector;
-- 조각 테이블: 768차원 임베딩 + 출처
create table doc_chunks (
  id bigserial primary key,
  doc_id text,           -- products/고구마.md
  content text,          -- 조각 본문
  embedding vector(768), -- 임베딩 차원과 일치
  metadata jsonb         -- {"section":"보관법"}
);
-- 근사 검색 인덱스 (코사인)
create index on doc_chunks
  using hnsw (embedding vector_cosine_ops);
```

▶ "doc_chunks 테이블과 hnsw 인덱스 SQL을 써 줘. 내가 SQL Editor에 붙여넣을게"

STEP 2 · 적재 스크립트 — scripts/ingest.ts

```
// md → ## 절로 자르기 → 임베딩 → 저장
const ai = new GoogleGenAI({}); // GEMINI_API_KEY
for (const file of mdFiles)         // Glob 결과
 for (const sec of
      splitBySection(file, 500, 50)) {
  const r = await ai.models.embedContent({
    model: "gemini-embedding-001",
    contents: sec.text,
    config: { outputDimensionality: 768 } });
  await supabase.from("doc_chunks").insert({
    doc_id: file, content: sec.text,
    embedding: r.embeddings[0].values,
    metadata: { section: sec.title } });
 }
```

▶ "docs/products/*.md 를 ## 절 단위로 잘라 임베딩하고 doc_chunks에 넣는 ingest.ts 를 써 줘"

STEP 3 · 검색 함수 — SQL, RPC로 호출

```
create or replace function match_chunks(
  query_embedding vector(768),
  match_count int default 5,      -- top-k
  match_threshold float default 0.5)
returns table(id bigint, doc_id text,
  content text, metadata jsonb,
  similarity float)
language sql stable as $$
  select c.id, c.doc_id, c.content, c.metadata,
    1 - (c.embedding <=> query_embedding)
      as similarity
  from doc_chunks c  -- 별칭 c: 반환 열 이름과 구분
  where 1 - (c.embedding <=> query_embedding)
        > match_threshold
  order by c.embedding <=> query_embedding
  limit match_count;
  -- <=> 는 코사인 거리 연산자
$$;
```

▶ "코사인 유사도로 상위 k개를 돌려주는 match_chunks 함수를 써 줘. 임계값 인자 포함"

※ 코드는 구조 설명용 최소 형태 — import · 오류 처리 · 형 선언 생략. SDK 이름(@google/genai · @supabase/supabase-js) · 메서드(embedContent · generateContent · rpc) · 2026.10 기준 · 수업 전 재확인

기술 구축 ③ — STEP 4~6, 서버 함수 · 화면 · 실행

질문이 들어오면 임베딩 → 검색 함수 → 찾은 조각만 근거로 생성 → 답과 출처를 화면에. 근거가 없으면 보류 문구가 먼저

STEP 4 · 서버 함수 — app/api/qa/route.ts

```
export async function POST(req: Request) {
  const { question } = await req.json();
  // STEP 2와 같은 임베딩 모델
  const q = await embed(question);
  const { data: chunks } = await supabase.rpc(
    "match_chunks", { query_embedding: q,
      match_count: 5, match_threshold: 0.5 });
  if (!chunks?.length)   // 근거 없음 → 보류
    return Response.json(
      { answer: HOLD, sources: [] });
  const ctx = chunks
    .map(c => `[${c.id}] ${c.content}`)
    .join("\n");
  const out = await ai.models.generateContent({
    model: "gemini-2.5-flash",
    contents: `아래 조각만 근거로 답하라.
      없으면 "보류".\n${ctx}\n`
        + `질문: ${question}` });
  return Response.json({ answer: out.text,
    sources: chunks.map(c => ({
      id: c.id, doc: c.doc_id,
      section: c.metadata.section })) });
}
```

▶ "질문 → 임베딩 → match_chunks → 조각만 근거로 Gemini 호출 → 답과 출처를 돌려주는 route.ts 를 써 줘. 키는 서버 환경 변수만"

STEP 5 · 화면 — 답 아래 출처

```
// app/products/[id]/qa.tsx (일부)
<p>{answer}</p>
<ul>
  {sources.map(s => (
    <li key={s.id}>출처: {s.doc} §{s.section}</li>
  ))}
</ul>
// 보류일 때는 sources 가 비어 있으므로
// 고객센터 안내 문구만 표시
```

▶ "답 아래에 출처 목록을 '출처: 파일 §절' 형식으로 보여 주고, 보류면 안내 문구만 보이게 해 줘"

STEP 6 · 실행 순서 → 확인은 다음 장

- **실행 순서** — SQL Editor(STEP 1 · 3) → `npx tsx scripts/ingest.ts` → `npm run dev` → 화면에서 질문
- **환경 변수** — `GEMINI_API_KEY` · `SUPABASE_URL` · `SUPABASE_SECRET_KEY`(Day 25의 서버 전용 키)는 `.env.local`, 커밋 금지(Day 25)
- **왜 서버에서만** — 서비스 키 · Gemini 키가 브라우저로 나가면 타인이 내 한도를 소진
- **시간 · 비용 감각** — 적재는 문서 10건에 수십 초, 질문 1회는 임베딩 1회 + 생성 1회(무료 등급 한도 안)
- **확인은 다음 장** — 행 수 · 검색 함수 · 화면 질문 2개 · 골든셋 적중률 순서로

※ 코드는 구조 설명용 최소 형태 — import · 오류 처리 · 형 선언 생략. SDK 이름(@google/genai · @supabase/supabase-js) · 메서드(embedContent · generateContent · rpc) · 2026.10 기준 · 수업 전 재확인

기술 구축 ④ — 동작 확인과 흔한 실패, 그리고 벡터 DB 없는 길

무엇을 실행해 어떤 결과가 나오면 된 것인가. 골든셋(Day 29 9장)의 질문 10개로 적중률을 재고, 문서가 50건 이하면 벡터 DB 없이 Claude Code 검색으로 같은 결과

실행할 것	이렇게 나오면 된 것
SQL Editor `select count(*) from doc_chunks`	문서 10건 × 절 3~5개 ≈ 30~50행. 0행이면 ingest 실패
RPC 직접 호출 `select * from match_chunks(…)`	질문 임베딩을 넣으면 similarity가 높은 순 5행, 0.5 미만은 제외
화면 질문 1 "고구마 보관법은?"	답 + "출처: products/고구마.md §보관법" — 출처를 열면 같은 내용
화면 질문 2 "주차장 요금은?"	보류 문구, sources 빈 배열. 지어낸 답이면 프롬프트 · 임계값 점검
골든셋 10문항 `quality/golden/qa/`	기대 출처와 실제 출처가 일치한 비율 = 적중률. `metrics.md`에 기록

단계	벡터 DB 길 (이 절)	벡터 DB 없는 길 — 문서 ≤ 50건 · 팀 내부
색인	ingest.ts → doc_chunks	없음 — md 절 제목이 색인
검색	match_chunks (코사인 상위 5)	Claude Code가 Grep(키워드) · Read(해당 절) — 동의어는 CLAUDE.md 규칙
생성	route.ts → Gemini	Claude Code 응답 + 근거 경로 규칙(8장)
사용자	앱 화면의 고객	팀원 (Claude Code 안에서만)

흔한 실패 세 가지

1. **청크가 너무 큼** — 절 하나가 2,000토큰이면 답이 흐려지고 비용 증가. 증상: 출처는 맞는데 답이 엉뚱. 조치: 500토큰 분할 확인
2. **임베딩 모델 · 차원 불일치** — 문서는 768, 질문은 3072로 만들면 RPC 오류 또는 similarity 전부 낮음. 조치: embed() 함수 하나를 양쪽에서 공유
3. **임계값** — 0.5가 너무 높으면 전부 보류, 너무 낮으면 엉뚱한 조각 포함. 조치: 골든셋으로 0.3~0.7 사이 비교

적중률 표 형식 — 질문 · 기대 출처 · 실제 출처 · 일치(O/X) · 보류 여부. 골든셋 원칙(작게 · 정답은 사람이 · 고정 · 회귀 확인)을 그대로 적용

실습 ① 설계 — 더미 데이터로 RAG 적용 전 · 후를 나란히 놓기

가상 상품 문서로 같은 질문 10개를 "문서 없이"(A)와 "문서를 근거로"(B) 두 번 답하게 해 비교. A는 빈칸을 업계 관행값으로 메우고(할루시네이션), B는 찾은 절만 말하고 못 찾으면 멈춤 — **틀리면 위험한 질문**에서 차이가 가장 큼

목적 **같은 질문, 두 가지 답** — A(적용 전)는 모델의 일반 지식만으로, B(적용 후)는 더미 문서를 검색해 근거로. 두 표를 나란히 놓으면 RAG가 무엇을 바꾸는지 설명 없이 보임왜 더미(가상) 데이터인가 — 실제 상품이면 모델이 어디선가 본 지식으로 맞힐 수 있어 차이가 흐려짐. **모델이 모를 수밖에 없는 사실**이어야 A와 B가 갈라짐

더미 데이터 설계 원칙 네 가지

- **모델이 모르는 사실** — 가상 상품명 · 상품 코드 · 가격 · 우리 가게만의 배송 정책. 검색해도 안 나오는 것
- **관행과 일부러 다른 값** — 무료 배송 45,000원 · 당일 배송은 달서구 · 북구만 · 사과칩은 냉장 필수. 모델이 관행("3만원 무료" "수성구 당일" "건조 간식은 상온")으로 답하면 틀리도록. 관행과 같은 값은 함정이 안 됨 (실제 실행에서 확인)
- **문서에 없는 질문도 준비** — 주차장 · 할인 쿠폰처럼 어느 문서에도 없는 질문 3개. 질문 파일에 유형(함정 · 없음)을 적지 않음 — 모델이 읽고 단서로 씀
- **작게** — 상품 10건 × 절 4개(설명 / 보관법 / 알레르기·원산지 / 배송). 절 제목이 곧 출처 표기 단위(6장)

"적용 전" = 문서가 손에 닿지 않는 상태 — Claude Code는 시키지 않아도 폴더를 검색해 근거를 찾음(9장). 그래서 A는 **문서를 저장소에 넣기 전**에 실행. 8장 규칙은 찾게 하는 장치가 아니라 **출처를 달고, 없으면 멈추게** 하는 규율

```
docs/dummy/products/P-0113_대구사과칩.md   ← 상품 10건 중 1건
# 대구사과칩 P-0113 (가상 상품)
## 설명          팔공산 사과 100% · 85g · 6,800원
## 보관법        개봉 후 반드시 냉장(상온 2일 변질) · 10일
## 알레르기 · 원산지   견과류 생산 라인 공유 — 주의
## 배송          달서구 · 북구 당일, 수성구 포함 그 외 익일
                 45,000원 이상 무료, 미만 3,900원
```

질문 유형	수	예	기대 결과 (A → B)
문서에 답이 있음	5	보관법 · 배송비 · 당일 배송 지역 · 용량 · 가격	관행값 추정 → 문서 값 + 출처
상식과 다른 함정	2	"상온에 두고 먹어도 되나요?" · "견과류 알레르기가 있어도 괜찮나요?"	**상식형 오답** → 문서대로
문서에 없음	3	주차장 요금 · 할인 쿠폰 · 제조 공장 주소	"알 수 없음"+관행 추정 → **보류**

※ 더미 데이터는 `강의자료/보충자료/보충01_RAG_실습데이터/`에 배포(상품 10건 · 질문 10개 · 표현 바꾼 질문 5개 · 강사용 정답 키 · 실습용 CLAUDE.md). 실습 전용 — 실제 데이터와 섞지 않음. 상품명 · 수치는 전부 가상

실습 ① 실행 — STEP 1~5, 문서 생성 → A 적용 전 → B 적용 후 → 비교표

A와 B는 **새 세션**에서 따로. 더미 문서와 8장 규칙은 **A를 끝낸 뒤**에 — Claude Code는 시키지 않아도 폴더를 검색하므로 문서가 있으면 A가 이미 RAG가 되고, 규칙이 먼저면 A도 보류함. 코드는 쓰지 않음

STEP 1 사람

질문 파일만 먼저

배포 폴더에서 `research/qa_dummy_questions.md`만 팀 저장소에 복사. **docs/dummy/는 아직 넣지 않음**

STEP 2 사람

정답 열 채우기

배포 폴더(저장소 밖)의 문서를 보고 "문서 정답" 열 기입

STEP 3 Claude Code

A · 적용 전

문서도 규칙도 없는 상태 · 새 세션 · 프롬프트 ② → `qa_dummy_A.md`

STEP 4 Claude Code

문서 넣기 · 규칙 · B

docs/dummy/ 복사(직접 만들면 프롬프트 ①) → 8장 규칙 네 줄을 `CLAUDE.md`에 → 새 세션 · 프롬프트 ③ → `qa_dummy_B.md`

STEP 5 사람

비교표 판정

A · B를 정답과 대조해 O / X / 보류 → 다음 장

프롬프트 ① — 더미 문서를 직접 만들 때 (STEP 4 · 배포본 대신)

```
docs/dummy/products/ 에 가상 식재료 상품 10개의 md 파일을 만들어 줘. 파일명 상품코드_상품명.md, 절은 ## 설명 / ## 보관법 / ## 알레르기 · 원산지 / ## 배송.
상품명 · 코드 · 가격 · 정책은 지어내되 서로 모순 없게.
관행과 다르게 — 사과칩은 "개봉 후 냉장 필수(상온 2일 변질)", "견과류 생산 라인 공유". 배송 절은 "달서구 · 북구만 당일, 45,000원 이상 무료, 미만 3,900원"으로 통일.
```

프롬프트 ② — A · 적용 전 (STEP 3 · 새 세션)

```
research/qa_dummy_questions.md 파일 하나만 읽어 줘. 다른 파일은 열거나 검색하지 말고, 네가 아는 일반 지식만으로 질문 10개에 답해 줘.
모르는 것은 가장 그럴듯한 답을 쓰고 답마다 확신도(상 · 중 · 하)를 붙여 research/qa_dummy_A.md 에 질문 · 답 · 확신도 표로 저장.
```

프롬프트 ③ — B · 적용 후 (STEP 4 · 새 세션)

```
CLAUDE.md 의 "노트 질문 답변 규칙"(8장)을 지켜 줘.
이번 근거 폴더는 docs/dummy/products/ — 그 안의 파일만 근거로 research/qa_dummy_questions.md 의 질문 10개에 답해 줘.
질문 · 답 · 근거 경로와 절 · 검색어 · 보류 여부 표로 research/qa_dummy_B.md 에 저장. 근거가 없으면 "보류".
```

완료 기준 — ① 더미 문서 10건 · 관행과 다른 값 3개 이상 ② 질문 10개에 "문서 정답" 열(없음 3개 표시) ③ A · B 각 10행, 다른 세션 — 문서 · 규칙은 A 뒤 · B 앞 ④ B 답마다 근거 경로 또는 "보류" · **산출물** — `research/qa_dummy_questions.md` · `qa_dummy_A.md` · `qa_dummy_B.md` · `docs/dummy/`

※ 막혔을 때 — A가 "모르겠다"만 하면 "가장 그럴듯한 답을 적고 확신도 '하'로". A가 보류하거나 문서 내용을 맞히면 규칙 · 문서가 먼저 들어간 것 — 빼고 새 세션. A에서 Grep · Glob 줄이 보이면 9장의 자동 검색(빈손) — 토의 거리. B가 근거 없이 답하면 "근거 없는 답은 보류로"

실습 ① 결과 읽기 — 비교표에서 보이는 세 가지 차이

`research/qa_dummy_compare.md` — 2026-10-31 실제 실행 결과 중 5행(원문은 보충자료 폴더). 실행마다 문장은 달라지나 양상은 같음 — 적중률보다 **틀리는 방식**을 읽는 것이 목적

질문	문서 정답	A · 적용 전 (일반 지식) — 실제 실행	B · 적용 후 (문서 근거) — 실제 실행	A / B
배송비가 무료가 되는 주문 금액은?	45,000원 이상 (미만 3,900원)	"보통 3만 원 이상 무료배송이 가장 흔하다" (하)	45,000원 이상 무료, 미만 3,900원 — P-0113 › 배송	X / O
수성구는 당일 배송이 되나요?	아니오 — 달서구 · 북구만 당일	"대구 시내이므로 당일 권역일 가능성" (하)	안 됨 — 당일은 달서구 · 북구, 수성구는 익일 — › 배송	X / O
대구사과칩을 상온에 두고 먹어도 되나요?	아니오 — 상온 2일 변질, 냉장 필수	"개봉 후 밀봉하면 상온에 두어도 무방" (중)	안 됨 — 상온 2일 안에 변질, 냉장 필수 — › 보관법	X / O
견과류 알레르기가 있는데 먹어도 되나요?	주의 — 견과류 생산 라인 공유	"사과만으로 만드는 것이 보통... 표시 확인" (중)	라인 공유 주의 — 가능 여부는 문서가 판단 안 함 — › 알레르기	△ / O
매장 주차장 요금은?	문서에 없음	"구매 고객 무료 주차가 많다... 알 수 없음" (하)	보류 — 매장 · 주차 내용 없음 (검색어: 주차 · 매장 · 픽업)	X / 보류

1 관행으로 메움, 틀린 줄 모름

A는 빈칸을 두지 않음 — "보통 3만 원", "30g 내외", "수성구는 가능성"처럼 업계 관행값으로 채움. 확신도는 낮게 달지만 수치 5개가 전부 틀렸고, 모델은 그것을 알 길이 없음(8장). 실습에서 볼 것 — "일반적으로"가 붙은 답은 어디서 온 값인가

2 출처가 검증 가능

B의 답은 "파일 › 절"을 달고 나옴 — 사람이 그 절을 열어 10초 안에 맞는지 확인 가능. 맞고 틀림보다 **확인할 수 있는가**가 바뀐 것.
실습에서 볼 것 — 출처 5개를 실제로 열어 일치 확인

3 모르면 멈춤

문서에 없는 3문항에서 B는 보류 + 확인한 검색어를 남김. A는 "알 수 없음"이라면서도 추정을 덧붙임(무료 주차 · 신규 쿠폰). 보류는 "다음에 문서에 추가할 것" 목록(8장)

기록 — `modeling/metrics.md`에 두 줄: **A 정답률 · B 정답률(보류 제외)**, 그리고 "틀리면 위험한 질문"에서의 A/B. 숫자는 각 팀의 실제 결과만

※ 비교표 프롬프트는 보충자료 절차서 부록 A · 실행 기록: A는 Read · Write만(3턴), B는 문서 10건 Read + Grep 2회(17턴) — Claude Code 2.1.314, 2026-10-31

실습 ② — 같은 더미 데이터로 키워드 검색 vs 벡터 검색 전 · 후

실습 ①의 B는 키워드 검색(Grep)이었음. 질문의 **표현을 바꾸면** 키워드는 놓치고 벡터는 찾는 순간이 보임 — 12~14장 구축을 더미 데이터로 돌리는 선택 실습(Day 25 이후)

STEP 1 Claude Code
더미 문서 적재
12장 STEP 1~3 그대로, 대상만 `docs/dummy/products/*.md` → `doc_chunks`

STEP 2 사람
표현 바꾼 질문 5개
배포된 `research/qa_dummy_paraphrase.md`(아래 표 왼쪽 열)

STEP 3 Claude Code
두 방식 실행
① Grep 키워드 검색 ② `match_chunks` 벡터 검색 — 찾은 절 · 유사도를 표로

STEP 4 사람
판정 · 전환 조건 결론
어느 질문에서 갈렸나 → 9장의 "벡터 DB가 필요해지는 조건"에 우리 경우 대입

표현을 바꾼 질문	문서의 표현	Grep · 키워드 — 실제 실행	match_chunks · 벡터 — 기대 (미검증)
사과칩을 오래 두고 먹으려면 어떻게 해야 하나요?	## 보관법	**불가 — "사과칩"은 제목만 일치**	**§보관법 → 냉장 필수 · 10일**
땅콩 때문에 걱정인데 먹어도 되나요?	견과류 생산 라인 공유	**불가 — "땅콩" · "걱정" 0건**	**§알레르기 · 원산지 → 주의**
오늘 시키면 오늘 오나요? (달서구)	당일 배송	**가능 — "달서구"가 §배송과 일치**	**§배송 → 당일(오후 2시 이전)**
P-0113 용량이 몇 그램이죠?	P-0113 · 85g	**파일 특정 — 숫자는 열어야 보임**	**§설명 → 85g (코드는 키워드가 정확, 7장)**
선물 포장 되나요?	(문서에 없음)	**0건 → 보류**	**임계값 미만 → 보류 (14장)**

프롬프트 — STEP 3

```
research/qa_dummy_paraphrase.md 의 질문 5개를
두 방식으로 각각 검색해 줘.
① Grep: 질문의 핵심 단어로 docs/dummy/products/ 검색
② match_chunks: 질문 임베딩 → 상위 3개 (임계값 0.5)
   — scripts/search.ts 로 실행(embed 재사용)
질문 · 방식 · 찾은 파일 §절 · 유사도 · 답 가능 여부 표로
research/qa_dummy_search.md 에 저장.
동의어 규칙 없이 비교.
```

완료 기준 — ① 5문항 × 2방식 = 10행 ② 키워드만 실패한 질문 2개 이상 표시 ③ "없는 질문"이 두 방식 모두 보류 ④ 결론 한 줄 — 우리 문서 수 · 사용자 · 속도에서 벡터 전환이 필요한가(9장)

※ 크리에이티브 트랙은 `worldview.md` · brand 스킬 문서로, 벡터 대신 CLAUDE.md "동의어 규칙"(땅콩 → 견과류) 전 · 후 비교 · Grep 열은 2026-10-31 실제 실행, 벡터 열은 기대값(Supabase · Gemini 환경에서 검증 후 추가)

실습 ③ — 앱 안 "상품 Q&A" 기능을 spec.md 한 단락으로

데이터·서비스 트랙 선택 실습(Day 25 이후). 더미 문서 대신 **실제 상품** 설명 · FAQ md 10건을 저장소 삼아 근거 표기 · 보류 규칙이 든 기능 명세를 작성. 구현은 다음 단계

STEP 1 Claude Code

문서 10건 틀 생성

`products` 테이블의 상품 10개에 대해 `docs/products/<상품>.md` — 설명 · 보관법 · FAQ 3문항 절. 내용은 사람이 확인 · 수정

›

STEP 2 Claude Code

파일 검색으로 시험

고객 질문 5개(보관 · 알레르기 · 원산지 · 배송 · 없는 정보 1개)에 8장 규칙(실습 ①의 B와 같은 방식)으로 답 → 근거 · 보류 확인

›

STEP 3 사람

명세 항목 결정

검색 단위(절) · 검색 방식(키워드 → 벡터 전환 조건) · 근거 표기 형식 · 보류 문구 · 금지 내용

›

STEP 4 Claude Code

spec.md 단락 작성

아래 프롬프트로 `spec.md`에 §상품 Q&A 추가. 사람은 Day 25 13장 설계 규칙 네 가지와 대조

프롬프트 예시 — STEP 4

```
spec.md 에 "상품 Q&A" 기능 단락을 추가해 줘. 항목:
1. 입력 — 상품 상세 화면의 질문 입력창 (200자 이내)
2. 문서 저장소 — docs/products/*.md 의 ## 절 단위
3. 검색 — 1차는 상품명 + 키워드 일치. 문서가 100건을
   넘거나 질문 표현이 다양해지면 Supabase pgvector로 전환
4. 생성 — 서버 함수에서만 호출(Day 25 규칙), 찾은 절만 프롬프트에
5. 근거 표기 — 답 아래 "출처: <상품>.md §<절 제목>"
6. 보류 — 근거 없으면 "등록된 상품 정보에 없는 내용입니다.
   고객센터로 문의해 주세요"로 고정, 효능 · 원산지 추정 금지
7. 로그 — 질문 · 찾은 절 · 보류 여부를 qa_log 테이블에 저장
docs/products/ 의 실제 절 제목을 예시로 넣어 줘.
```

완료 기준

- [ ] `docs/products/*.md` 10건, 절 제목이 질문 단위(보관 · 알레르기 …)
- [ ] STEP 2 시험 5문항 중 "없는 정보" 1개가 보류로 처리
- [ ] `spec.md` §상품 Q&A에 일곱 항목 + 벡터 전환 조건 명시
- [ ] Day 25 설계 규칙 4개(키는 서버 · 형식 고정 · 근거 제한 · 실패 대비) 반영

산출물 — `docs/products/*.md` · `spec.md` §상품 Q&A · `research/qa_check.md` (STEP 2 기록, 실습 ① B 표 형식)

AX 크리에이티브 트랙 — 같은 절차로 `worldview.md` · brand 스킬 문서를 저장소 삼아 "세계관 Q&A"(설정 · 금칙어 · 톤) 단락을 `spec.md` 에

막혔을 때 — 쓸 내용이 없으면 `products` 테이블 열로만 쓰고 나머지 절은 "미확인"

용어 카드 · 자가 점검 — RAG에서 Grounding까지 여섯 용어

채용 공고의 표현과 우리 과정의 대응물을 한 표에. 자가 점검의 답은 이 덱의 장 번호

용어	영문	한 줄 정의	채용 공고에서의 표현	우리 과정의 대응물
RAG	Retrieval-Augmented Generation	답하기 전에 관련 문서 조각을 검색해 입력에 덧붙이고 그 근거로 생성	"RAG 파이프라인 구축 · 운영"	노트 절 구조 + CLAUDE.md 규칙 + spec.md §상품 Q&A
청크	Chunk · Chunking	검색 · 임베딩 단위로 자른 문서 조각. 크기 · 겹침 · 메타데이터	"청킹 전략 설계"	`research/*.md`의 `##` 절
임베딩	Embedding	텍스트를 의미가 보존되는 숫자 벡터로 바꾼 것	"임베딩 모델 선정 · 파인튜닝"	Day 19 7장 원리 · Day 4 리뷰 태그 묶음
벡터 DB	Vector Database	벡터를 저장하고 질문 벡터와 가까운 것을 빠르게 찾는 DB	"Pinecone · Chroma · pgvector 경험"	Supabase `pgvector` (앱 기능 시 선택)
검색	Retrieval · Retriever	질문에 맞는 청크 k개를 고르는 단계. 키워드 · 벡터 · 하이브리드	"검색 품질(recall) 개선"	Claude Code `Grep` · `Read` 호출, 서버 함수 DB 조회
그라운딩	Grounding(근거 고정) · Citation	답을 검색된 조각 안의 정보에만 묶고 출처를 표기, 없으면 보류	"환각(hallucination) 저감 · 출처 표기"	CLAUDE.md 네 줄 규칙 · 출처 등급 A~D · 보류 문구

1. **research/ 노트 200건을 질문마다 전부 넣으면 생기는 문제 세 가지는?**
   답 → 4장 (열화 · 컨텍스트 창 한도 · 근거 추적 불가, 그리고 비용)

2. **"상품 코드 P-0113이 들어간 노트"를 찾을 때 키워드 검색과 의미 검색 중 무엇이 맞고, 왜?**
   답 → 7장 (고유명사 · 코드는 키워드 검색의 강점)

3. **우리 앱에 벡터 DB가 필요해지는 조건 세 가지는?**
   답 → 9장 (문서 수천 건 · 앱 사용자가 질문 · 화면 응답 속도)

※ 정의는 Lewis 외 2020(arxiv 2005.11401) · LangChain RAG 문서 기준의 수업용 번역. 채용 공고 표현은 2026.10 국내 공고에서 흔한 문구의 예시이며 특정 공고 인용 아님